KONGU ENGINEERING COLLEGE

(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 11

PROJECT TITLE

WORKER HEALTH AND SAFETY MONITORING SYSTEM FOR CONSTRUCTION

SUBMITTED BY

ABISHEK R S - 24MER002

GOWRISANKAR K - 24MER010

NAVEENRAJ K - 24MER035

PRAVEEN N M - 24MER043

WORKER HEALTH AND SAFETY MONITORING SYSTEM FOR CONSTRUCTION

1. Project review:

The Worker Health and Safety Monitoring System for Construction is designed to monitor the safety condition of workers at construction sites. Construction workers may be exposed to hazardous conditions such as high temperature, harmful gases, accidents, and unsafe working environments.

The proposed system uses an **ESP8266 NodeMCU** as the central controller along with sensors such as a **DHT11 temperature and humidity sensor** and an **MQ2 gas sensor**. The system continuously monitors the surrounding environmental conditions.

When abnormal temperature, humidity, or harmful gas levels are detected, the ESP8266 activates a **buzzer and LED** to provide an immediate warning. The system can also be extended using Wi-Fi connectivity for IoT-based remote monitoring and emergency alerts.

2. Problem Statement:

Construction sites can expose workers to dangerous environmental conditions. High temperature, excessive humidity, and harmful gases may affect worker health and safety.

Manual monitoring of these conditions is difficult and may not provide immediate warnings during emergency situations. Failure to detect hazardous conditions at an early stage can result in accidents and health risks.

Therefore, an automatic system is required to continuously monitor worker safety conditions and provide an immediate alert when abnormal environmental conditions are detected.

The proposed system uses sensors, an ESP8266 NodeMCU, buzzer, and LED to monitor environmental conditions and provide a safety warning.

3. Proposed Solution:

The proposed solution consists of an ESP8266-based Worker Health and Safety Monitoring System.

The **DHT11 sensor** monitors temperature and humidity, while the **MQ2 gas sensor** detects the presence of harmful gases or smoke. The sensor readings are continuously sent to the ESP8266 NodeMCU.

The ESP8266 processes the sensor readings and compares them with predefined safety limits. If an abnormal temperature, humidity, or gas condition is detected, the system activates the buzzer and LED.

Proposed Operation:

1. Start the system and initialize the ESP8266.
2. Initialize the DHT11 and MQ2 sensors.
3. Measure the temperature and humidity.
4. Detect the gas or smoke level.
5. Send the sensor readings to the ESP8266.
6. Process and compare the readings with predefined safety limits.
7. Check whether a hazardous condition is detected.
8. If an abnormal condition occurs, activate the safety alert.
9. Turn ON the buzzer and LED.
10. Display the worker safety warning.
11. Continue monitoring the environment continuously.

4. System Architecture:

The Worker Health and Safety Monitoring System consists of the following sections:

Input Section:

DHT11 Sensor – Measures temperature and humidity.

MQ2 Gas Sensor – Detects gas and smoke levels.

Processing Section:

The ESP8266 NodeMCU acts as the central controller.

Receives sensor data.

Processes temperature, humidity, and gas readings.

Compares the readings with predefined safety limits.

Detects hazardous conditions.

Controls the buzzer and LED.

Actuation Section:

The buzzer and LED provide safety indications.

- Buzzer OFF and LED OFF → Safe condition.
- Buzzer ON and LED ON → Hazardous condition detected.

Connectivity Section:

The ESP8266 has built-in Wi-Fi capability.

The system can be extended for:

- IoT-based worker monitoring.
- Remote safety alerts.
- Cloud data storage.
- Mobile notifications.
- Construction site safety monitoring.

5. Circuit Table:

Important Pin Mapping:

ESP8266 NodeMCU Pin		Component
GPIO 2	D4	DHT11 Sensor → DATA
A0	A0	MQ2 Gas Sensor → AO
GPIO 12	D6	Buzzer → Positive Terminal
GPIO 13	D7	LED → Anode through resistor
3.3V / 5V	–	Sensor Power
GND	–	Common Ground

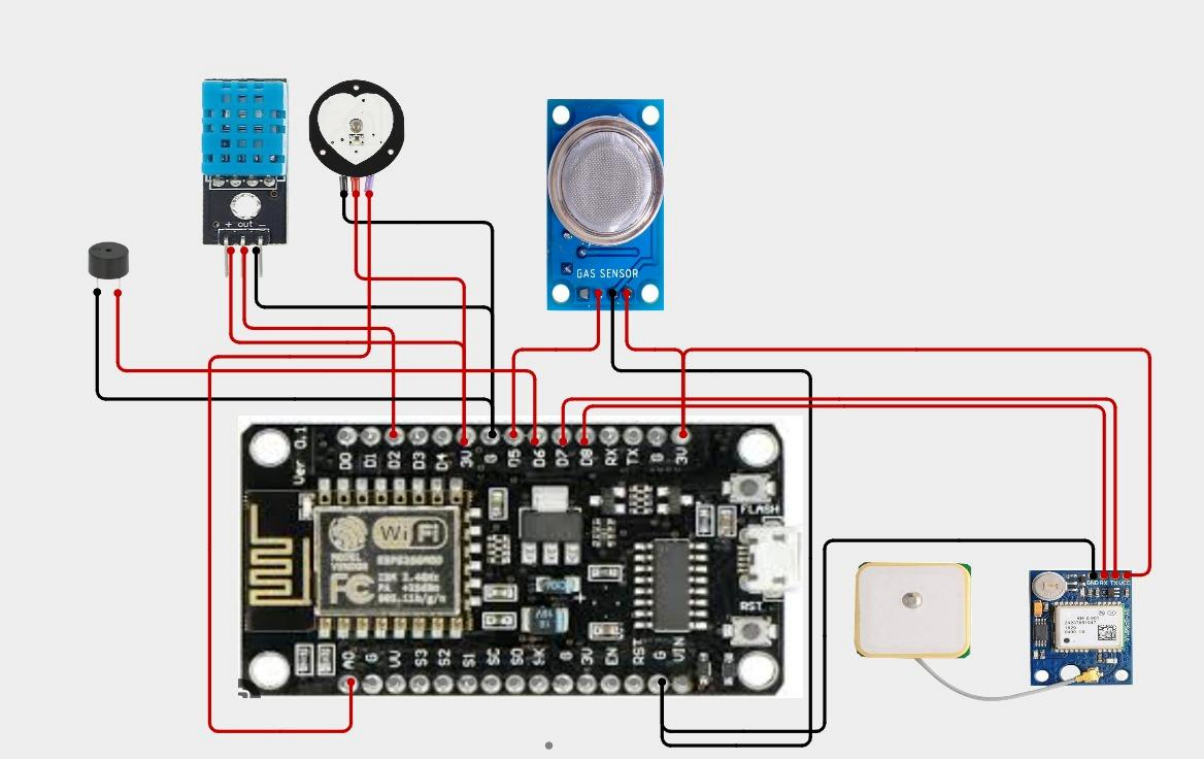
6. Circuit Design:

The circuit consists of an **ESP8266 NodeMCU, DHT11 sensor, MQ2 gas sensor, buzzer, LED, breadboard, and jumper wires**.

The DHT11 sensor DATA pin is connected to **D4 (GPIO2)** of the ESP8266 to measure temperature and humidity. The analog output of the MQ2 gas sensor is connected to the **A0 pin** to monitor gas or smoke levels.

The buzzer is connected to **D6 (GPIO12)**, and the LED is connected to **D7 (GPIO13)** through a suitable resistor.

The ESP8266 continuously reads the sensor values. When the temperature, humidity, or gas level exceeds the predefined safety limit, the controller activates the buzzer and LED to provide a warning.



7. Algorithm:

1. Start the system.
2. Initialize the **ESP8266 NodeMCU, DHT11 sensor, MQ2 gas sensor, buzzer, and LED**.
3. Read the **temperature value** from the DHT11 sensor.
4. Read the **humidity value** from the DHT11 sensor.
5. Read the **gas or smoke level** from the MQ2 sensor.
6. Compare all sensor readings with the predefined safety limits.
7. Check whether **high temperature, excessive humidity, or harmful gas** is detected.
8. If a hazardous condition is detected, turn **ON the buzzer and LED**.
9. Display **"HAZARDOUS CONDITION – WORKER SAFETY ALERT"**; otherwise, keep the buzzer and LED **OFF** and display **"SAFE CONDITION"**.
10. Wait for a short interval and **repeat the monitoring process continuously**.

8. Coding:

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>
#include <DHT.h>
#include <TinyGPS++.h>
#include <SoftwareSerial.h>

// ============================================================
// WORKER HEALTH AND SAFETY MONITORING SYSTEM
// ESP8266 NodeMCU
// Pulse Sensor + DHT11 + MQ2 + GPS + Buzzer
// WiFi + ThingSpeak
// ============================================================



// ============================================================
// WIFI SETTINGS
// ============================================================

const char* WIFI_SSID = "praveen";
const char* WIFI_PASSWORD = "praveen12345";



// ============================================================
// THINGSPEAK SETTINGS
// ============================================================
```

```
// Your ThingSpeak Write API Key
const char* API_KEY = "0IP4GOX0SMNK3JBV";

// Replace with your actual ThingSpeak Channel ID
unsigned long CHANNEL_ID = 1234567;



// ================================================================
// PIN DEFINITIONS
// ================================================================

// Pulse Sensor
#define PULSE_PIN A0

// DHT11
#define DHT_PIN 4
#define DHT_TYPE DHT11

// MQ-2 Digital Output
#define MQ2_PIN 14

// Buzzer
#define BUZZER_PIN 12

// GPS
// GPS TX -> ESP8266 D7 / GPIO13
// GPS RX -> ESP8266 D8 / GPIO15
```

```
#define GPS_RX_PIN 13
#define GPS_TX_PIN 15


// ═══════════════════════════════════════════════════════
// OBJECTS
// ═══════════════════════════════════════════════════════

DHT dht(DHT_PIN, DHT_TYPE);

TinyGPSPlus gps;

SoftwareSerial gpsSerial(
  GPS_RX_PIN,
  GPS_TX_PIN
);

WiFiClient client;


// ═══════════════════════════════════════════════════════
// SAFETY SETTINGS
// ═══════════════════════════════════════════════════════

float HIGH_TEMPERATURE = 40.0;

float HIGH_HUMIDITY = 85.0;
```

```
// Demonstration values only
int LOW_PULSE = 50;
int HIGH_PULSE = 120;


// ============================================================
// SENSOR VARIABLES
// ============================================================

int pulseValue = 0;

int heartRate = 0;

float temperature = 0.0;

float humidity = 0.0;

bool gasDetected = false;

bool unsafeCondition = false;


// ============================================================
// TIMERS
// ============================================================
```

```
unsigned long lastThingSpeakUpdate = 0;

const unsigned long UPDATE_INTERVAL = 20000;

unsigned long lastWiFiCheck = 0;

const unsigned long WIFI_CHECK_INTERVAL = 10000;



// ================================================================
// SETUP
// ================================================================

void setup()
{
  Serial.begin(9600);

  delay(1000);

  // DHT11
  dht.begin();

  // MQ2
  pinMode(MQ2_PIN, INPUT);

  // Buzzer
  pinMode(BUZZER_PIN, OUTPUT);
```

```
  digitalWrite(BUZZER_PIN, LOW);

  // GPS
  gpsSerial.begin(9600);

  Serial.println();
  Serial.println("═══════════════════════════════════════");
  Serial.println(" WORKER HEALTH & SAFETY MONITORING");
  Serial.println(" ESP8266 NODEMCU");
  Serial.println("═══════════════════════════════════════");

  delay(2000);

  // Connect WiFi
  connectWiFi();

  // Start ThingSpeak
  ThingSpeak.begin(client);

  Serial.println();
  Serial.println("ThingSpeak Initialized");
  Serial.println("System Ready");
  Serial.println("═══════════════════════════════════════");
}
```

```
// ============================================================
// WIFI CONNECTION
// ============================================================

void connectWiFi()
{
  Serial.println();
  Serial.println("Connecting to WiFi...");

  WiFi.mode(WIFI_STA);

  WiFi.begin(
    WIFI_SSID,
    WIFI_PASSWORD
  );

  int attempts = 0;

  while (
    WiFi.status() != WL_CONNECTED &&
    attempts < 30
  )
  {
    delay(500);

    Serial.print(".");
```

```
    attempts++;
  }

  Serial.println();

  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.println("WiFi CONNECTED!");

    Serial.print("IP Address: ");

    Serial.println(
      WiFi.localIP()
    );
  }
  else
  {
    Serial.println("WiFi CONNECTION FAILED!");
  }
}



// ============================================================
// READ GPS
// ============================================================

void readGPS()
```

```
{
  while (gpsSerial.available())
  {
    gps.encode(
      gpsSerial.read()
    );
  }
}


// ============================================================
// READ PULSE SENSOR
// ============================================================

void readPulse()
{
  pulseValue = analogRead(PULSE_PIN);

  // Simple demonstration logic.
  // This is NOT a medical-grade BPM calculation.

  if (pulseValue > 500)
  {
    heartRate = 80;
  }
  else
  {
```

```
    heartRate = 0;
  }
}


// ================================================================
// READ DHT11
// ================================================================

void readDHT()
{
  temperature = dht.readTemperature();

  humidity = dht.readHumidity();
}


// ================================================================
// READ MQ2
// ================================================================

void readMQ2()
{
  int gasState = digitalRead(MQ2_PIN);

  // Most MQ2 modules:
  // LOW  = gas detected
```

```
  // HIGH = normal

  if (gasState == LOW)
  {
    gasDetected = true;
  }
  else
  {
    gasDetected = false;
  }
}


// ================================================================
// SAFETY ANALYSIS
// ================================================================

void analyzeSafety()
{
  unsafeCondition = false;

  // Gas detection

  if (gasDetected)
  {
    unsafeCondition = true;
```

```
  Serial.println(
    "WARNING: GAS / SMOKE DETECTED"
  );
}

// High temperature

if (
  !isnan(temperature) &&
  temperature > HIGH_TEMPERATURE
)
{
  unsafeCondition = true;

  Serial.println(
    "WARNING: HIGH TEMPERATURE"
  );
}

// High humidity

if (
  !isnan(humidity) &&
  humidity > HIGH_HUMIDITY
)
{
  unsafeCondition = true;
```

```
  Serial.println(
    "WARNING: HIGH HUMIDITY"
  );
}

// Low pulse

if (
  heartRate > 0 &&
  heartRate < LOW_PULSE
)
{
  unsafeCondition = true;

  Serial.println(
    "WARNING: LOW PULSE RATE"
  );
}

// High pulse

if (
  heartRate > HIGH_PULSE
)
{
  unsafeCondition = true;
```

```
    Serial.println(
      "WARNING: HIGH PULSE RATE"
    );
  }

  // Buzzer

  if (unsafeCondition)
  {
    digitalWrite(
      BUZZER_PIN,
      HIGH
    );
  }
  else
  {
    digitalWrite(
      BUZZER_PIN,
      LOW
    );
  }
}


// ================================================================
// PRINT GPS
```

```
// ================================================================

void printGPS()
{
 if (gps.location.isValid())
 {
  Serial.print("Latitude         : ");

  Serial.println(
   gps.location.lat(),
   6
  );

  Serial.print("Longitude        : ");

  Serial.println(
   gps.location.lng(),
   6
  );
 }
 else
 {
  Serial.println(
   "GPS              : Searching..."
  );
 }
```

```
  if (gps.satellites.isValid())
  {
    Serial.print(
      "GPS Satellites    : "
    );

    Serial.println(
      gps.satellites.value()
    );
  }
  else
  {
    Serial.println(
      "GPS Satellites    : Searching..."
    );
  }
}



// ================================================================
// PRINT SENSOR STATUS
// ================================================================

void printStatus()
{
  Serial.println();
  Serial.println(
```

```
  "-----------------------------------"
);

Serial.print(
  "Pulse Sensor Value : "
);

Serial.println(
  pulseValue
);

Serial.print(
  "Heart Rate        : "
);

if (heartRate > 0)
{
  Serial.print(heartRate);
  Serial.println(" BPM");
}
else
{
  Serial.println(
    "No pulse detected"
  );
}
```

```
// Temperature

if (!isnan(temperature))
{
  Serial.print(
    "Temperature       : "
  );

  Serial.print(
    temperature,
    1
  );

  Serial.println(" C");
}
else
{
  Serial.println(
    "Temperature       : DHT ERROR"
  );
}

// Humidity

if (!isnan(humidity))
{
  Serial.print(
```

```
    "Humidity          : "
   );

   Serial.print(
    humidity,
    1
   );

   Serial.println(" %");
}
else
{
   Serial.println(
    "Humidity          : DHT ERROR"
   );
}

// Gas

if (gasDetected)
{
   Serial.println(
    "Gas Status        : DANGER"
   );
}
else
{
```

```
  Serial.println(
    "Gas Status        : NORMAL"
  );
}

// Worker safety

if (unsafeCondition)
{
  Serial.println(
    "Worker Status     : UNSAFE"
  );

  Serial.println(
    "Buzzer            : ON"
  );
}
else
{
  Serial.println(
    "Worker Status     : SAFE"
  );

  Serial.println(
    "Buzzer            : OFF"
  );
}
```

```
  // WiFi

  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.println(
      "WiFi             : CONNECTED"
    );
  }
  else
  {
    Serial.println(
      "WiFi             : DISCONNECTED"
    );
  }

  // GPS

  printGPS();

  Serial.println(
    "-----------------------------------"
  );
}


// ================================================================
```

```
// SEND DATA TO THINGSPEAK
// ============================================================

void sendToThingSpeak()
{
 if (WiFi.status() != WL_CONNECTED)
 {
  Serial.println(
   "WiFi OFF - Upload skipped"
  );

  return;
 }

 Serial.println();
 Serial.println(
  "Uploading data to ThingSpeak..."
 );


 // ============================================================
 // FIELD 1 - PULSE SENSOR
 // ============================================================

 ThingSpeak.setField(
  1,
  (int)pulseValue
```

```
);



// ============================================================
// FIELD 2 - TEMPERATURE
// ============================================================

ThingSpeak.setField(
  2,
  (float)temperature
);



// ============================================================
// FIELD 3 - HUMIDITY
// ============================================================

ThingSpeak.setField(
  3,
  (float)humidity
);



// ============================================================
// FIELD 4 - GAS STATUS
// ============================================================
```

```
int gasStatus;

if (gasDetected)
{
  gasStatus = 1;
}
else
{
  gasStatus = 0;
}

ThingSpeak.setField(
  4,
  gasStatus
);



// ================================================================
// FIELD 5 - WORKER SAFETY
// ================================================================

int safetyStatus;

if (unsafeCondition)
{
  safetyStatus = 1;
}
```

```
else
{
  safetyStatus = 0;
}

ThingSpeak.setField(
  5,
  safetyStatus
);



// ================================================================
// FIELD 6 - GPS LATITUDE
// ================================================================

if (gps.location.isValid())
{
  // IMPORTANT:
  // Convert double to float to avoid
  // ThingSpeak setField ambiguity.

  float latitude =
    (float)gps.location.lat();

  ThingSpeak.setField(
    6,
    latitude
```

```
  );
}
else
{
  ThingSpeak.setField(
    6,
    (float)0.0
  );
}


// ================================================================
// FIELD 7 - GPS LONGITUDE
// ================================================================

if (gps.location.isValid())
{
  // IMPORTANT:
  // Convert double to float.

  float longitude =
    (float)gps.location.lng();

  ThingSpeak.setField(
    7,
    longitude
  );
```

```
}
else
{
  ThingSpeak.setField(
    7,
    (float)0.0
  );
}


// ================================================================
// SEND ALL FIELDS
// ================================================================

int response =
  ThingSpeak.writeFields(
    CHANNEL_ID,
    API_KEY
  );


// ================================================================
// CHECK RESPONSE
// ================================================================

if (response == 200)
{
```

```
    Serial.println(
      "ThingSpeak upload SUCCESS!"
    );
  }
  else
  {
    Serial.print(
      "ThingSpeak ERROR: "
    );

    Serial.println(
      response
    );
  }
}



// ================================================================
// CHECK WIFI
// ================================================================

void checkWiFi()
{
  if (
    millis() -
    lastWiFiCheck >=
    WIFI_CHECK_INTERVAL
```

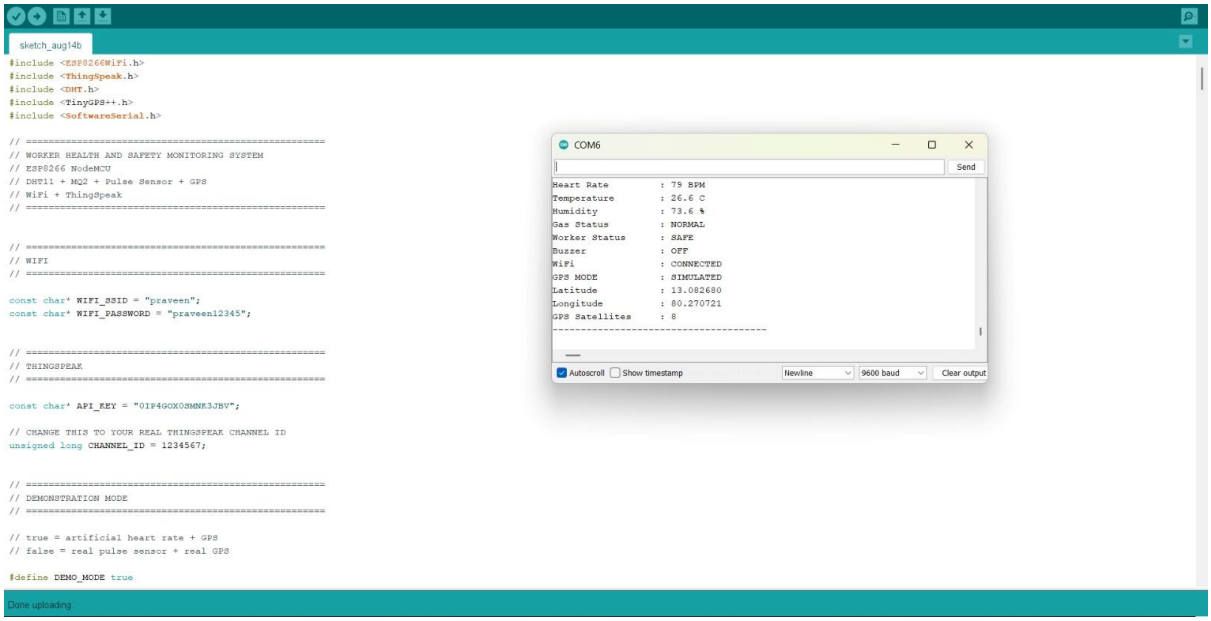
```
  )
  {
    lastWiFiCheck = millis();

    if (
      WiFi.status() != WL_CONNECTED
    )
    {
      Serial.println();
      Serial.println(
        "WiFi disconnected!"
      );

      connectWiFi();
    }
  }
}



// ============================================================
// MAIN LOOP
// ============================================================

void loop()
{
  // ------------------------------------------------
  // Read GPS continuously
```

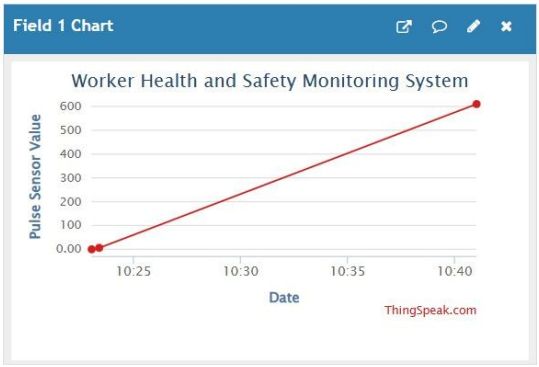
```
// ------------------------------------------------

readGPS();



// ------------------------------------------------
// Read sensors
// ------------------------------------------------

readPulse();

readDHT();

readMQ2();



// ------------------------------------------------
// Analyze worker safety
// ------------------------------------------------

analyzeSafety();



// ------------------------------------------------
// Display status
// ------------------------------------------------
```

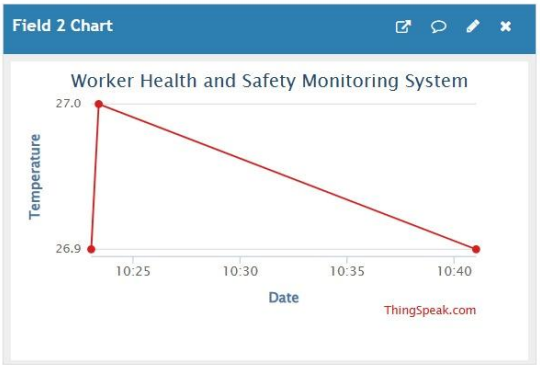
```
printStatus();


// ------------------------------------------------
// Check WiFi
// ------------------------------------------------

checkWiFi();


// ------------------------------------------------
// Send to ThingSpeak every 20 seconds
// ------------------------------------------------

if (
  millis() -
  lastThingSpeakUpdate >=
  UPDATE_INTERVAL
)
{
  lastThingSpeakUpdate = millis();

  if (
    !isnan(temperature) &&
    !isnan(humidity)
  )
  {
```

```
    sendToThingSpeak();
  }
 }


 delay(3000);
}
```

Coding Output:



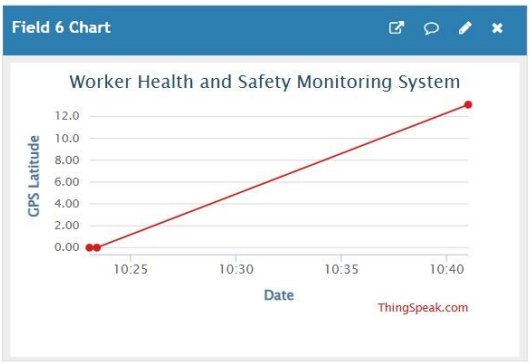

Field 1 Chart
Worker Health and Safety Monitoring System
Pulse Sensor Value
600
500
400
300
200
100
0.00
10:25
10:30
10:35
10:40
Date
ThingSpeak.com


Field 2 Chart
Worker Health and Safety Monitoring System
Temperature
27.0
26.9
10:25
10:30
10:35
10:40
Date
ThingSpeak.com


Field 3 Chart
Worker Health and Safety Monitoring System
Humidity
73.1
73.0
72.9
72.8
72.7
10:25
10:30
10:35
10:40
Date
ThingSpeak.com


Field 4 Chart
Worker Health and Safety Monitoring System
Gas Status
0.00
10:25
10:30
10:35
10:40
Date
ThingSpeak.com


Field 5 Chart
Worker Health and Safety Monitoring System
Worker Safety
0.00
10:25
10:30
10:35
10:40
Date
ThingSpeak.com


Field 6 Chart
Worker Health and Safety Monitoring System
GPS Latitude
12.0
10.0
8.00
6.00
4.00
2.00
0.00
10:25
10:30
10:35
10:40
Date
ThingSpeak.com


Field 7 Chart
Worker Health and Safety Monitoring System
GPS Longitude
80.0
60.0
40.0
20.0
0.00
10:25
10:30
10:35
10:40
Date
ThingSpeak.com

9. System Working:

Power ON the System:

The ESP8266 NodeMCU is powered ON.

Initialize Components:

The DHT11 sensor, MQ2 gas sensor, buzzer, and LED are initialized.

Monitor Temperature and Humidity

The DHT11 sensor continuously measures the temperature and humidity

Monitor Gas Level

The MQ2 sensor continuously detects gas or smoke levels

Process Sensor Data

The ESP8266 receives and processes the sensor readings.

Check Safety Condition

The sensor readings are compared with predefined safety limits.

Hazardous Condition

high temperature, excessive humidity, or gas is detected:

- Buzzer → ON
- LED → ON
- Safety Alert → Activated
- Worker Status → Warning

Safe Condition

If all environmental conditions are within the predefined limits:

- Buzzer → OFF
- LED → OFF
- Worker Status → Safe

Repeat the Process:

- The system continuously monitors the construction site environment and checks for hazardous conditions.

10. Prototype Implementation:

The prototype is implemented using an **ESP8266 NodeMCU, DHT11 sensor, MQ2 gas sensor, buzzer, LED, breadboard, and jumper wires**.

The DHT11 sensor monitors the surrounding temperature and humidity, while the MQ2 sensor detects harmful gases or smoke. The ESP8266 processes the sensor data and determines whether the environment is safe or hazardous.

Hardware Setup

Connect the DHT11 DATA pin → D4 (GPIO2).

Connect the MQ2 sensor analog output → A0.

Connect the Buzzer → D6 (GPIO12).

Connect the LED → D7 (GPIO13) through a suitable resistor.

Connect the sensor power pins to a suitable power supply.

Connect all GND terminals to a common ground.

Upload the program to the ESP8266 using Arduino IDE.

Power ON the prototype.

Observe the temperature, humidity, and gas readings.

Create an abnormal condition for testing.

Observe the activation of the buzzer and LED.

Monitor the worker safety status through the Serial Monitor.

Prototype Working:

The ESP8266 continuously monitors environmental conditions.

The DHT11 measures temperature and humidity.

The MQ2 sensor monitors gas or smoke levels.

The sensor readings are processed by the ESP8266.

The values are compared with predefined safety limits.

If a hazardous condition is detected, the buzzer is activated.

The LED provides a visual warning.

If all conditions are normal, the system indicates a safe working environment.

The process repeats continuously.

11. Bill of Materials:

S.No.	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	DHT11 Sensor	Temperature and Humidity Sensor	1
3	MQ2 Sensor	Gas and Smoke Detection Sensor	1
4	Buzzer	3.3V/5V Buzzer	1
5	LED	Standard Indicator LED	1
6	Resistor	Suitable LED Current Limiting Resistor	1
7	Breadboard	Standard Prototype Board	1
8	Jumper Wires	Male-Male / Male-Female	1 Set
9	USB Cable	Micro-USB	1
10	Power Supply	Suitable Regulated Supply	1

12. Results & Performance Analysis:

The prototype successfully demonstrates the basic operation of a worker health and safety monitoring system for construction sites.

- The DHT11 sensor continuously monitors temperature and humidity.
- The MQ2 sensor detects gas and smoke levels.
- The ESP8266 successfully processes the sensor readings.

- The system automatically identifies safe and hazardous environmental conditions.
- When an abnormal condition is detected, the buzzer provides an audible warning.
- The LED provides a visual safety indication.
- The system can be extended with IoT connectivity for remote worker safety monitoring and emergency notifications.

Results

The DHT11 sensor successfully measures temperature and humidity.

The MQ2 sensor successfully detects gas and smoke levels.

The ESP8266 successfully processes the sensor readings.

The system continuously monitors the construction environment.

Hazardous conditions are automatically detected.

The buzzer provides an audible safety alert.

The LED provides a visual warning.

The system identifies safe and unsafe working conditions.

The prototype demonstrates the basic operation of a **Worker Health and Safety Monitoring System for Construction**.

Performance Analysis:

Parameter	Expected Performance
Temperature Monitoring	Continuous
Humidity Monitoring	Continuous
Gas Monitoring	Continuous
Data Processing	Real-time
Hazard Detection	Automatic
Safety Alert	Buzzer ON
Visual Warning	LED ON
Safe Condition	Buzzer and LED OFF